



Déploiement d'une architecture 5G et analyse du protocole PFCP

Déploiement 5G

Noms / Prénoms : GUEYE Mame Bousso, MOLONGO–ARMOUDON Justine,
GUEYE Codou, LE PABIC Ronan, Amaury MOUTIER

Encadré par : M.OUAMRI

Formation : BUT Réseaux & Télécommunications - Parcours Réseaux Opérateurs &
Multimédias

SOMMAIRE

SOMMAIRE.....	2
INTRODUCTION.....	4
Présentation de l'architecture.....	4
Rôle des composants et des protocoles.....	5
Principaux composants.....	6
Principaux protocoles et interfaces.....	7
Déroulement simplifié d'une connexion.....	8
Configuration du réseau des VMs.....	8
Déploiement et configuration d'Ueransim.....	9
1. Objectif de la manipulation.....	9
2. Vérification de la configuration réseau de la machine.....	10
Partie 1 - Installation et configuration d'UERANSIM.....	11
1. Modification des fichiers UE et gNB.....	12
Ouvrir le fichier de configuration du gNB :.....	12
2. Vérifier la prise en charge des interfaces TUN.....	13
Partie 2 - Installation et configuration de Open5GS CP.....	14
1. Objectif de la manipulation.....	14
2. Vérification de la configuration réseau de la machine.....	15
3. Installer les dépendances nécessaires à Open5GS.....	16
4. Compiler et installer Open5GS.....	16
5. Installer et vérifier MongoDB.....	18
6. Modification des fichiers SMF, AMF et NRF.....	19
a. Configurer le NRF.....	19
b. Configurer l'AMF.....	20
c. Configurer l'accès au NRF.....	21
d. Configurer l'interface N2.....	21
e. Configurer le PLMN et le TAC.....	21
f. Configurer le slice autorisé.....	22
g. Configurer le SMF.....	23
h. Configurer le serveur SBI.....	23
i. Configurer l'accès au NRF.....	24
j. Configurer PFCP.....	24
k. Configurer le réseau attribué aux UE.....	24
7. Autres fonctions nécessaires.....	26
8. Installer et lancer la WebUI Open5GS.....	26
Partie 3 — Configuration de la machine Open5GS UPF.....	29
1. Objectif de la manipulation.....	29

2. Vérification de la configuration réseau de la machine.....	30
3. Installer les dépendances nécessaires à Open5GS.....	31
4. Compiler et installer Open5GS.....	31
5. Modification des fichiers UPF.....	33
a. Configurer l'UPF.....	33
b. Explication de la partie PFCP.....	34
c. Explication de la partie GTP-U.....	35
d. Explication du réseau des UE.....	35
6. Créer l'interface virtuelle ogstun.....	35
7. Configurer le transfert IPv4.....	37
Partie 4 — Mise en service et validation d'une session utilisateur 5G.....	38
1. Objectif de la manipulation.....	38
2. Démarrer MongoDB.....	38
3. Démarrer les fonctions du plan de contrôle.....	39
Terminal CP 1 — NRF.....	39
Terminal CP 2 — UDR.....	39
Terminal CP 3 — UDM.....	40
Terminal CP 4 — AUSF.....	40
Terminal CP 5 — PCF.....	40
Terminal CP 6 — NSSF.....	40
Terminal CP 7 — SMF.....	41
Terminal CP 8 — AMF.....	41
4. Préparer et démarrer l'UPF.....	41
5. Vérifier l'association PFCP entre le SMF et l'UPF.....	43
6. Démarrer le gNB.....	44
7. Démarrer le UE.....	45
8. Enregistrement du UE.....	45
9. Création de la session PDU.....	45
10. Confirmer l'enregistrement dans les journaux de l'AMF.....	46
11. Vérifier l'adresse IP et la connectivité du UE.....	46
CONCLUSION TECHNIQUE.....	53
CONCLUSION GÉNÉRALE.....	54
Références.....	55

INTRODUCTION

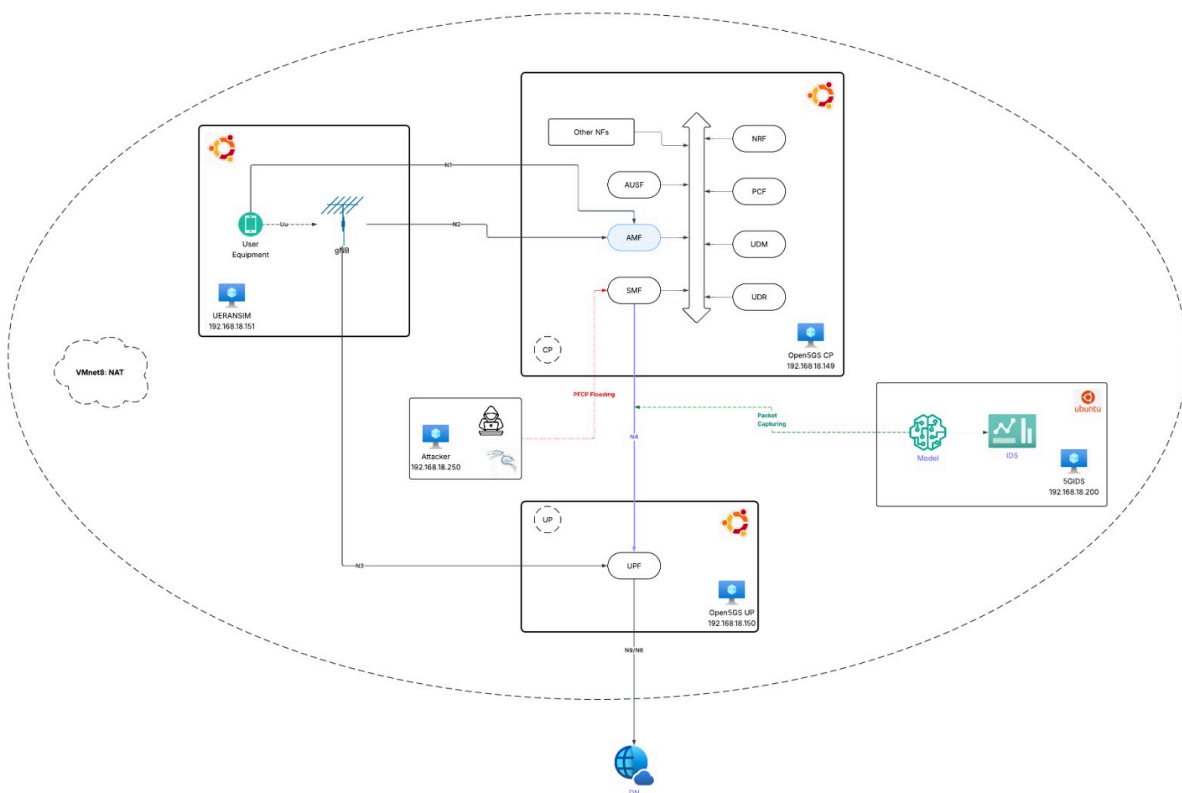
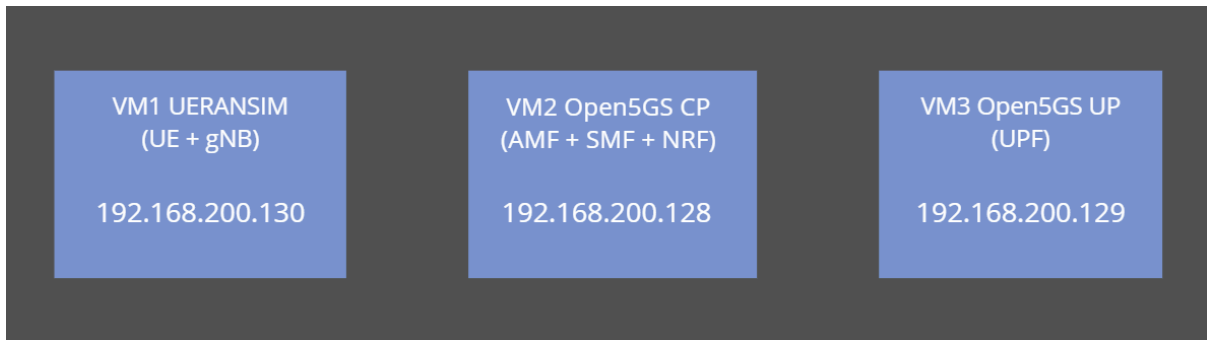
Ce document présente le déploiement d'une infrastructure 5G simplifiée, la validation d'une session utilisateur et l'analyse du protocole PFCP. Les risques de sécurité associés à PFCP sont ensuite présentés de manière théorique.

L'architecture déployée repose sur trois machines virtuelles principales :

- **UERANSIM** : Simulation des équipements utilisateur (UE) et de la station de base 5G (gNB)
- **Open5GS CP** : Cœur de réseau 5G avec les fonctions de contrôle (AMF, SMF, UPF central)
- **Open5GS UP** : Plan utilisateur avec l'UPF edge et la passerelle vers le réseau de données

Cette infrastructure permet de reproduire fidèlement le comportement d'un réseau 5G réel tout en offrant un environnement contrôlé pour tester différentes attaques sur l'interface N4 (PFCP) et collecter les données nécessaires.

Présentation de l'architecture



Rôle des composants et des protocoles

L'architecture mise en place reproduit de manière simplifiée le fonctionnement d'un réseau 5G. UERANSIM simule le terminal utilisateur, appelé UE, ainsi que la station de base 5G, appelée gNB. Le gNB sert d'intermédiaire entre l'UE et le cœur de réseau Open5GS. Le plan de contrôle, hébergé sur la machine Open5GS CP, prend en charge l'authentification, l'enregistrement de l'abonné et la création de sa session de données. Le plan utilisateur, représenté par l'UPF, transporte ensuite les paquets de l'UE. Les différentes fonctions communiquent à travers plusieurs interfaces et protocoles spécialisés..

Principaux composants

Composant	Rôle
UE	Représente le téléphone, un modem ou le terminal de l'abonné. Il demande à s'enregistrer dans le réseau et à obtenir une session de données.
gNB	Représente la station de base 5G. Il relaie la signalisation de l'UE vers l'AMF et transporte les données utilisateur vers l'UPF.
AMF	Gère la connexion du gNB, l'enregistrement de l'UE, la mobilité et les messages de signalisation NAS.
SMF	Gère les sessions PDU. Il sélectionne l'UPF, attribue les paramètres de la session et programme l'UPF avec PFCP.
UPF	Transporte les données de l'UE. Il applique les règles reçues du SMF et assure le passage entre le tunnel GTP-U et le réseau de données.
NRF	Joue le rôle d'annuaire du cœur 5G. Les fonctions réseau s'y enregistrent afin de pouvoir se découvrir.

Composant	Rôle
AUSF	Participe à l'authentification de l'abonné.
UDM	Gère les informations d'identité et d'abonnement de l'UE.
UDR	Récupère dans MongoDB les informations nécessaires sur l'abonné.
PCF	Fournit les politiques appliquées à la session, notamment les paramètres de qualité de service.
NSSF	Vérifie et sélectionne le slice demandé par l'UE. Dans cette architecture, le slice utilisé est SST 1.
MongoDB	Stocke le profil de l'abonné : IMSI, clés d'authentification, slice et DNN autorisés.
WebUI Open5GS	Permet d'ajouter ou de modifier les abonnés présents dans MongoDB. Elle ne participe pas directement aux échanges 5G.

Principaux protocoles et interfaces

Interface	Communication	Protocole	Rôle
Interface radio simulée	UE ↔ gNB	RRC	Établit et gère la connexion entre l'UE et la gNB. UERANSIM simule cette liaison sans utiliser de véritable transmission radio.
N1	UE ↔ AMF	NAS	Transporte les messages d'enregistrement, d'authentification et de gestion de session de l'UE. Ces messages passent physiquement par la gNB.
N2	gNB ↔ AMF	NGAP sur SCTP 38412	Transporte la signalisation entre le réseau d'accès et le cœur 5G. Cette interface permet notamment à la gNB de transmettre les messages NAS de l'UE à l'AMF.

Interface	Communication	Protocole	Rôle
N3	gNB ↔ UPF	GTP-U sur UDP 2152	Transporte les données réelles de l'UE dans un tunnel entre la gNB et l'UPF.
N4	SMF ↔ UPF	PFCEP sur UDP 8805	Permet au SMF de créer, modifier et supprimer les sessions dans l'UPF.
SBI	Fonctions du plan de contrôle	HTTP/2	Permet aux fonctions AMF, SMF, NRF, AUSF, UDM, UDR, PCF et NSSF d'échanger des informations et d'utiliser leurs services respectifs.

Déroulement simplifié d'une connexion

Lorsqu'un UE démarre, il établit d'abord une connexion RRC avec la gNB. L'UE envoie ensuite une demande d'enregistrement NAS, que le gNB transmet à l'AMF par l'interface N2. L'AMF utilise l'AUSF, l'UDM et l'UDR afin de vérifier les informations de l'abonné stockées dans MongoDB. Après l'authentification, l'AMF accepte l'enregistrement de l'UE.

L'UE demande ensuite une session PDU pour le DNN internet. L'AMF transmet cette demande au SMF. Le SMF programme l'UPF grâce au protocole PFCEP sur l'interface N4. Il lui transmet notamment les règles permettant de reconnaître et d'acheminer les paquets de l'UE. Une fois la session créée, l'UE reçoit une adresse IP et ses données sont transportées entre le gNB et l'UPF dans un tunnel GTP-U sur l'interface N3.

Configuration du réseau des VMs

Name	Type	External Connection	Host Connection	DHCP	Subnet Address
VMnet0	Custom	-	-	Enabled	192.168.200.0
VMnet1	Host-only	-	Connected	Enabled	192.168.233.0
LAN SiteA	Host-only	-	Connected	-	192.168.1.0

Ueransim

▶ Power on this virtual machine

▶ Edit virtual machine settings

▼ Devices

Memory

7.5 GB

Processors

4

Hard Disk (SCSI)

40 GB

CD/DVD (SATA)

Using file C:\User...

Network Adapter

NAT

Network Adapter 2

Custom (VMnet0)

USB Controller

Present

Sound Card

Auto detect

Display

Auto detect

open5gscp

▶ Power on this virtual machine

▶ Edit virtual machine settings

▼ Devices

Memory

7.5 GB

Processors

4

Hard Disk (SCSI)

32 GB

CD/DVD (SATA)

Using file C:\User...

Network Adapter

NAT

Network Adapter 2

Custom (VMnet0)

USB Controller

Present

Sound Card

Auto detect

Display

Auto detect

Open5gsup

▶ Power on this virtual machine

▶ Edit virtual machine settings

▼ Devices

Memory

7.5 GB

Processors

4

Hard Disk (SCSI)

40 GB

CD/DVD (SATA)

Using file C:\User...

Network Adapter

NAT

Network Adapter 2

Custom (VMnet0)

USB Controller

Present

Sound Card

Auto detect

Display

Auto detect

Déploiement et configuration d'Ueransim

1. Objectif de la manipulation

Cette partie a pour objectif de mettre en place la machine virtuelle UERANSIM chargée de simuler deux équipements d'un réseau 5G :

- un équipement utilisateur, appelé **UE** ;
- une station de base 5G, appelée **gNB**.

Le gNB doit pouvoir établir une connexion avec l'AMF du cœur Open5GS par l'interface N2. Il doit également communiquer avec l'UPF par l'interface N3 afin de transporter les données de l'UE.

UERANSIM ne reproduit pas une véritable transmission radio. Il simule les échanges entre l'UE et le gNB ainsi que les principaux protocoles nécessaires au raccordement avec le cœur 5G.

Les communications utilisées sont les suivantes :

Communication	Protocole	Destination
UE vers gNB	RRC et NAS simulés	gNB UERANSIM
gNB vers AMF	NGAP sur SCTP 38412	Open5GS CP
gNB vers UPF	GTP-U sur UDP 2152	Open5GS UPF

2. Vérification de la configuration réseau de la machine

Afficher le nom de la machine et ses interfaces réseau :

```
Shell  
hostname
```

```
Shell  
ip -br address
```

Vérifier que l'interface utilisée pour le réseau 5G possède l'adresse suivante :

192.168.200.130/24

Dans notre architecture, cette adresse est configurée sur l'interface **ens37**.

Tester ensuite la communication avec les deux autres machines :

```
Shell  
ping -c 2 192.168.200.128
```

```
Shell  
ping -c 2 192.168.200.129
```

Partie 1 – Installation et configuration d'UERANSIM

Installer les paquets nécessaires à la compilation :

Shell

```
sudo apt update && sudo apt install -y cmake make gcc g++  
libsctp-dev lksctp-tools iproute2 git
```

Se placer dans le dossier **/opt**, puis récupérer les sources d'Ueransim :

Shell

```
cd /opt
```

Shell

```
sudo git clone https://github.com/aligungr/UERANSIM
```

Shell

```
cd UERANSIM
```

Compiler le projet :

Shell

```
sudo cmake -B build
```

Shell

```
sudo cmake --build build
```

Vérifier que les exécutables nécessaires ont bien été générés :

```
Shell
ls -lh /opt/UERANSIM/build/nr-gnb
ls -lh /opt/UERANSIM/build/nr-ue
```

Cela sont les deux exécutables à lancer lorsque nous ferons les simulations.

1. Modification des fichiers UE et gNB

Ouvrir le fichier de configuration du gNB :

```
Shell
sudo nano /opt/UERANSIM/config/open5gs-gnb.yaml
```

Configurer les paramètres principaux :

```
GNU nano 8.4 /opt/UERANSIM/config/open5gs-gnb.yaml
mcc: '999' # Mobile Country Code value
mnc: '70' # Mobile Network Code value (2 or 3 digits)

nci: '0x000000010' # NR Cell Identity (36-bit)
idLength: 32 # NR gNB ID length in bits [22...32]
tac: 1 # Tracking Area Code

linkIp: 192.168.200.130 # gNB's local IP address for Radio Link Simulation (Usually same with local IP)
ngapIp: 192.168.200.130 # gNB's local IP address for N2 Interface (Usually same with local IP)
gtpIp: 192.168.200.130 # gNB's local IP address for N3 Interface (Usually same with local IP)

# List of AMF address information
amfConfigs:
  - address: 192.168.200.128
    port: 38412

# List of supported S-NSSAIs by this gNB
slices:
  - sst: 1

# Indicates whether or not SCTP stream number errors should be ignored.
ignoreStreamIds: true

# Cell access type. When set to one of the satellite types (nr-leo, nr-meo,
# nr-geo, nr-othersat), the gNB attaches the NR-NTN TAI Information extension
# to every UserLocationInformationNR it sends to the AMF. Defaults to "nr".
cellAccessType: nr
```

Ouvrir le fichier de configuration du UE :

```
Shell
sudo nano /opt/UERANSIM/config/open5gs-ue.yaml
```

Configurer les paramètres principaux :

```
GNU nano 8.4 /opt/UERANSIM/config/open5gs-ue.yaml
# IMSI number of the UE. IMSI = [MCC|MNC|MSISDN] (In total 15 digits)
supi: 'imsi-999700000000001'
# Mobile Country Code value of HPLMN
mcc: '999'
# Mobile Network Code value of HPLMN (2 or 3 digits)
mnc: '70'
# SUCI Protection Scheme : 0 for Null-scheme, 1 for Profile A and 2 for Profile B
protectionScheme: 0
# Home Network Public Key for protecting with SUCI Profile A
homeNetworkPublicKey: '5a8d38864820197c3394b92613b20b91633cbd897119273bf8e4a6f4eec0a650'
# Home Network Public Key ID for protecting with SUCI Profile A
homeNetworkPublicKeyId: 1
# Routing Indicator
routingIndicator: '0000'

# Permanent subscription key
key: '465B5CE8B199B49FAA5F0A2EE238A6BC'
# Operator code (OP or OPC) of the UE
op: 'E8ED289DEBA952E4283B54E88E6183CA'
# This value specifies the OP type and it can be either 'OP' or 'OPC'
opType: 'OPC'
# Authentication Management Field (AMF) value
amf: '8000'
# IMEI number of the device. It is used if no SUPI is provided
imei: '356938035643803'
# IMEISV number of the device. It is used if no SUPI and IMEI is provided
imeiSv: '4370816125816151'

# Network mask used for the UE's TUN interface to define the subnet size
tunNetmask: '255.255.255.0'

# List of gNB IP addresses for Radio Link Simulation
gnbSearchList:
- 192.168.200.130
```

```
ueransim@ueransim:~/UERANSIM$ ls ./build/
libdevbnd.so nr-binder nr-cli nr-gnb nr-ue
ueransim@ueransim:~/UERANSIM$
```

2. Vérifier la prise en charge des interfaces TUN

Vérifier la présence du périphérique TUN :

```
Shell
ls -l /dev/net/tun
```

Ce périphérique sera utilisé par UERANSIM pour créer l'interface réseau du UE après l'établissement de la session PDU.

Vérifier également la présence du fichier suivant :

```
Shell
ls -l /etc/iproute2/route_tables
```

Si le fichier est absent, le créer :

```
Shell
sudo mkdir -p /etc/iproute2
```

```
Shell
sudo tee /etc/iproute2/route_tables >/dev/null <<'EOF'
255 local
254 main
253 default
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
EOF
```

Cette manipulation évite l'erreur suivante :

***TUN configuration failure
Could not open '/etc/iproute2/route_tables'***

Partie 2 - Installation et configuration de Open5GS CP

1. Objectif de la manipulation

Mettre en place la machine virtuelle chargée d'héberger le **plan de contrôle du cœur 5G Open5GS**.

Cette machine devra assurer :

- l'enregistrement du gNB auprès du cœur 5G ;
- l'authentification du UE ;

- la vérification des informations de l'abonné dans MongoDB ;
- la gestion de la mobilité et de l'enregistrement ;
- la création et la gestion des sessions PDU ;
- la communication PFCP avec l'UPF.

Les principales fonctions installées sur cette machine sont les suivantes :

Fonction	Rôle
NRF	Répertorie les fonctions du cœur 5G
AMF	Gère l'enregistrement, la mobilité et la signalisation de l'UE
SMF	Crée et contrôle les sessions PDU
AUSF	Participe à l'authentification de l'UE
UDM	Gère les données et les identités des abonnés
UDR	Accède aux informations stockées dans MongoDB
PCF	Applique les politiques réseau
NSSF	Sélectionne le slice demandé par l'UE
MongoDB	Stocke les profils des abonnés

Le démarrage des différentes fonctions sera effectué dans une partie ultérieure, après la configuration de l'ensemble des machines.

2. Vérification de la configuration réseau de la machine

Afficher le nom de la machine et ses interfaces réseau :

```
Shell
hostname
```

```
Shell
ip -br address
```

Vérifier que l'interface utilisée pour les communications 5G possède l'adresse suivante :

192.168.200.128/24

Dans notre architecture, cette adresse est configurée sur l'interface ens37.

Tester ensuite la communication avec les deux autres machines :

Shell

```
ping -c 2 192.168.200.129
```

Shell

```
ping -c 2 192.168.200.130
```

3. Installer les dépendances nécessaires à Open5GS

Installer les outils de compilation et les bibliothèques requises :

Shell

```
sudo apt update
```

Shell

```
sudo apt install -y python3-pip python3-setuptools python3-wheel \
    ninja-build build-essential flex bison git cmake libsctp-dev \
    libgnutls28-dev libgcrypt-dev libssl-dev libidn11-dev \
    libmongoc-dev \
    libbson-dev libyaml-dev libnghttp2-dev libmicrohttpd-dev \
    libcurl4-gnutls-dev libtins-dev libtallic-dev meson
```

4. Compiler et installer Open5GS

Se placer dans le dossier contenant les sources d'Open5GS :

Shell

```
cd ~/open5gs-2.7.7
```


Créer le dossier de compilation avec Meson :

Shell

```
meson setup build --prefix="$(pwd)/install"
```

Compiler et installer Open5GS :

Shell

```
ninja -C build
```

Shell

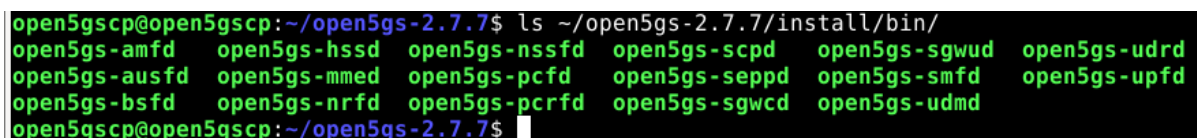
```
ninja -C build install
```

Vérifier que les exécutables ont bien été générés :

Shell

```
ls -lh ~/open5gs-2.7.7/install/bin/open5gs-*
```

Les exécutables utilisés pour le plan de contrôle doivent notamment être présents :



```
open5gscp@open5gscp:~/open5gs-2.7.7$ ls ~/open5gs-2.7.7/install/bin/
open5gs-amfd  open5gs-hssd  open5gs-nssfd  open5gs-scpd  open5gs-sgwud  open5gs-udrd
open5gs-ausfd open5gs-mmed  open5gs-pcfd  open5gs-seppd open5gs-smfd  open5gs-upfd
open5gs-bsfd  open5gs-nrfd  open5gs-pcrfd  open5gs-sgwcd open5gs-udmd
open5gscp@open5gscp:~/open5gs-2.7.7$
```

Vérifier également la présence des fichiers de configuration :

Shell

```
ls -lh ~/open5gs-2.7.7/install/etc/open5gs/*.yaml
```

Les fichiers réellement utilisés se trouvent dans :

~/open5gs-2.7.7/install/etc/open5gs/

Ne pas modifier les fichiers présents dans :

~/open5gs-2.7.7/build/configs/open5gs/

Ces derniers correspondent aux fichiers utilisés pendant la compilation et ne sont pas nécessairement ceux chargés lors du lancement des fonctions.

5. Installer et vérifier MongoDB

MongoDB est utilisé pour stocker les informations des abonnés, notamment :

- l'IMSI ;
- la clé d'authentification ;
- l'OPc ;
- le paramètre AMF ;
- le slice autorisé ;
- le DNN autorisé.

Installer les paquets nécessaires :

```
Shell
sudo apt update
```

```
Shell
sudo apt install -y gnupg curl
```

Importer la clé publique du dépôt MongoDB :

```
Shell
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
  sudo gpg --yes -o /usr/share/keyrings/mongodb-server-7.0.gpg
--dearmor
```

Ajout du dépôt MongoDB :

```
Shell
echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg
arch=amd64 ] \
  https://repo.mongodb.org/apt/debian bookworm/mongodb-org/7.0
main" | \
  sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
```

Mettre à jour la liste des paquets, puis installer MongoDB :

```
Shell  
sudo apt update
```

```
Shell  
sudo apt install -y mongodb-org
```

Démarrer le service et activer son démarrage automatique :

```
Shell  
sudo systemctl start mongod
```

```
Shell  
sudo systemctl status mongod | grep Active
```

Vérifier que le service est actif :

```
Shell  
systemctl is-active mongod
```

Vérifier également que MongoDB écoute sur le port 27017 :

```
Shell  
sudo ss -lntp | grep 27017
```

MongoDB peut écouter uniquement sur 127.0.0.1, car l'UDR et la base de données sont installés sur la même machine.

6. Modification des fichiers SMF, AMF et NRF

a. Configurer le NRF

Ouvrir le fichier de configuration du NRF :

```
Shell  
sudo nano ~/open5gs-2.7.7/install/etc/open5gs/nrf.yaml
```

Vérifier que le serveur SBI du NRF utilise l'adresse suivante :

```
XML
nrf:
  sbi:
    server:
      - address: 127.0.0.10
        port: 7777
```

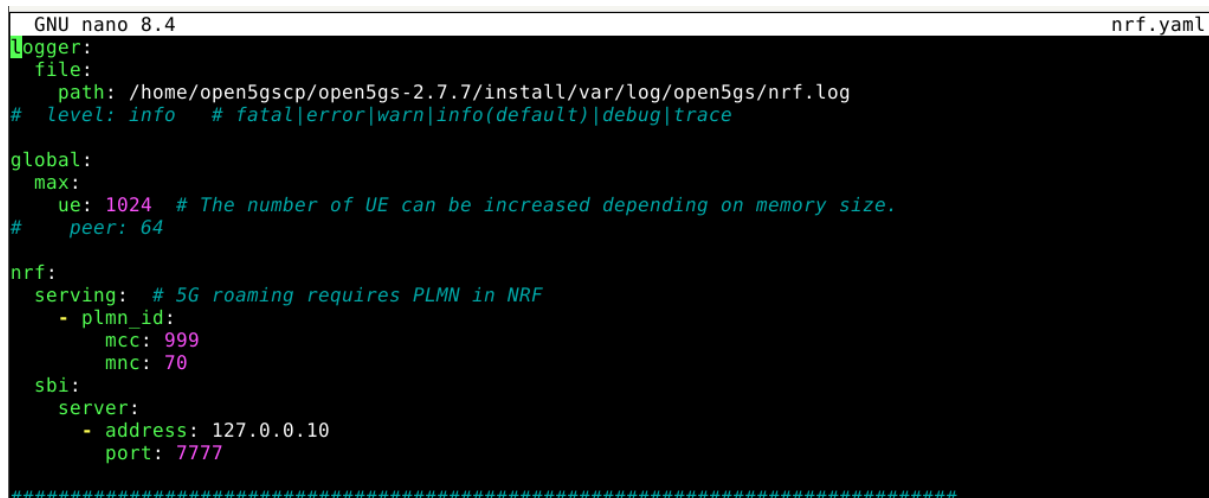
Le NRF est l'annuaire du cœur 5G.

Les autres fonctions Open5GS s'enregistrent auprès de lui afin de pouvoir se découvrir et communiquer entre elles. C'est pour cela qu'il est impératif de lancer le NRF en premier lieu avant les autres services.

Dans cette architecture, le NRF est accessible à l'adresse :

<http://127.0.0.10:7777>

L'adresse de boucle locale peut être utilisée, car toutes les fonctions du plan de contrôle sont installées sur la même machine.



```
GNU nano 8.4 nrf.yaml
logger:
  file:
    path: /home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/nrf.log
# level: info # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024 # The number of UE can be increased depending on memory size.
# peer: 64

nrf:
  serving: # 5G roaming requires PLMN in NRF
    - plmn_id:
        mcc: 999
        mnc: 70
  sbi:
    server:
      - address: 127.0.0.10
        port: 7777
#####
```

b. Configurer l'AMF

Ouvrir le fichier de configuration de l'AMF :

```
Shell
sudo nano ~/open5gs-2.7.7/install/etc/open5gs/amf.yaml
```

c. Configurer l'accès au NRF

Vérifier que l'AMF utilise directement le NRF :

```
XML
client:
  nrf:
    - uri: http://127.0.0.10:7777
```

Ne pas utiliser le NRF comme un SCP.

La section suivante ne doit pas être active :

```
XML
#   scp:
#   - uri: http://127.0.0.10:7777
```

Le NRF et le SCP remplissent des rôles différents.

d. Configurer l'interface N2

Configurer l'adresse utilisée par l'AMF pour recevoir la connexion du gNB :

```
XML
ngap:
  server:
    - address: 192.168.200.128
```

L'AMF utilisera cette adresse sur l'interface N2.

NGAP est transporté par SCTP sur le port : 38412

Le gNB UERANSIM devra donc contacter :

192.168.200.128:38412

e. Configurer le PLMN et le TAC

Vérifier les paramètres du réseau mobile simulé :

```
XML
tai:
  - plmn_id:
      mcc: 999
      mnc: 70
      tac: 1
```

Les valeurs doivent correspondre à celles définies dans les fichiers UERANSIM :

MCC = 999

MNC = 70

TAC = 1

f. Configurer le slice autorisé

Configurer le slice accepté par l'AMF :

```
XML
plmn_support:
  - plmn_id:
      mcc: 999
      mnc: 70
  s_nssai:
    - sst: 1
```

Dans cette manipulation, le slice est défini uniquement par :

SST = 1

Ne pas ajouter de valeur SD si elle n'est pas configurée dans UERANSIM et dans le profil de l'abonné.

Une incohérence entre le slice demandé par l'UE et celui autorisé par l'AMF pourrait provoquer :

***Cannot find Requested NSSAI
Registration reject***

L'AMF assure principalement :

- la connexion du gNB ;
- la réception des messages NAS du UE ;
- l'enregistrement du UE ;
- la gestion de la mobilité ;
- la sélection des fonctions nécessaires à l'authentification ;

- le transfert de la demande de session vers le SMF.

```
GNU nano 8.4 amf.yaml
logger:
  file:
    path: /home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/amf.log
# level: info # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024 # The number of UE can be increased depending on memory size.
# peer: 64

amf:
  sbi:
    server:
      - address: 192.168.200.128
        port: 7779
    client:
# nrf:
#   - uri: http://127.0.0.10:7777
    scp:
      - uri: http://127.0.0.10:7777
  ngap:
    server:
      - address: 192.168.200.128
  metrics:
    server:
      - address: 192.168.200.128
        port: 9090
  guami:
    - plmn_id:
        mcc: 999
        mnc: 70
      amf_id:
        region: 2
        set: 1
  tai:
    - plmn_id:
        mcc: 999
        mnc: 70
      tac: 1
```

g. Configurer le SMF

Ouvrir le fichier de configuration du SMF :

```
Shell
sudo nano ~/open5gs-2.7.7/install/etc/open5gs/smf.yaml
```

h. Configurer le serveur SBI

Vérifier que le serveur SBI du SMF utilise :

```
XML
smf:
  sbi:
    server:
      - address: 192.168.200.128
```

port: 7777

Cette interface permet notamment à l'AMF de demander la création ou la modification d'une session PDU.

i. Configurer l'accès au NRF

Vérifier que le SMF utilise le NRF :

```
XML
client:
  nrf:
    - uri: http://127.0.0.10:7777
```

j. Configurer PFCP

Configurer la communication entre le SMF et l'UPF :

```
XML
pfcf:
  server:
    - address: 192.168.200.128
  client:
    upf:
      - address: 192.168.200.129
```

Cette configuration indique que :

- le SMF utilise l'adresse **192.168.200.128** pour PFCP ;
- l'UPF à contrôler utilise l'adresse **192.168.200.129** ;
- la communication utilise **UDP** sur le port **8805**.

Cette liaison correspond à l'interface N4.

Le SMF utilise PFCP pour créer, modifier et supprimer les sessions dans l'UPF.

k. Configurer le réseau attribué aux UE

Vérifier la configuration du réseau des abonnés :

XML

session:

- subnet: 10.45.0.0/16
- gateway: 10.45.0.1
- dnn: internet

Le réseau **10.45.0.0/16** sera utilisé pour attribuer une adresse IP à l'UE.

La passerelle **10.45.0.1** correspondra à l'interface **ogstun** présente sur la machine UPF.

Le DNN utilisé par le UE est : internet

Le SMF assure principalement :

- la création des sessions PDU ;
- la sélection de l'UPF ;
- la gestion de l'adresse IP du UE ;
- la programmation de l'UPF avec PFCP ;
- la création des règles PDR, FAR et QER.

```
GNU nano 8.4 /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/smf.yaml
file:
  path: /home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/smf.log
# level: info # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024 # The number of UE can be increased depending on memory size.
    peer: 64

smf:
  sbi:
    server:
      - address: 192.168.200.128
        port: 7777
    client:
      nrf:
        - uri: http://127.0.0.10:7777
      scp:
        - uri: http://192.168.200.128:7778
  pfcf:
    server:
      - address: 192.168.200.128
    client:
      upf:
        - address: 192.168.200.129
  gtpc:
    server:
      - address: 192.168.200.128
  gtpu:
    server:
      - address: 192.168.200.128
  metrics:
    server:
      - address: 192.168.200.128
        port: 9090
  session:
    - subnet: 10.45.0.0/16
      gateway: 10.45.0.1
    - subnet: 2001:db8:cafe::/48
      gateway: 2001:db8:cafe::1
```

7. Autres fonctions nécessaires

Les fonctions suivantes doivent également être présentes lors du démarrage du cœur 5G :

Fonction	Rôle
AUSF	Participe à l'authentification du UE
UDM	Gère les informations d'identité et d'abonnement
UDR	Récupère les informations stockées dans MongoDB
PCF	Fournit les politiques appliquées aux sessions
NSSF	Participe à la sélection du slice

Dans cette manipulation, leurs fichiers de configuration par défaut sont conservés.

Lors de l'authentification, la chaîne utilisée est notamment :

AMF

→ AUSF

→ UDM

→ UDR

→ MongoDB

Le NSSF permet de vérifier que le slice demandé par l'UE, ici SST 1, est disponible.

Le PCF fournit les politiques utilisées pendant la création de la session.

8. Installer et lancer la WebUI Open5GS

La WebUI permet de créer et de modifier les abonnés présents dans MongoDB.

Installer **Node.js** et npm si nécessaire :

```
Shell
sudo apt install -y nodejs npm
```

Vérifier leur présence :

```
Shell
node --version
```

```
Shell  
npm --version
```

Se placer dans le dossier de la WebUI :

```
Shell  
cd ~/open5gs-2.7.7/webui
```

Installer les dépendances si elles ne sont pas déjà présentes :

```
Shell  
npm install
```

Lancer ensuite la WebUI :

```
Shell  
npm run dev
```

Depuis le navigateur de la machine Open5GS CP, ouvrir :

<http://localhost:9999>

La WebUI est uniquement un outil d'administration de MongoDB.

Elle ne participe pas directement aux échanges réseau entre le UE, le gNB, l'AMF, le SMF et l'UPF.

Elle n'a pas besoin de rester ouverte après l'ajout de l'abonné.

A login form for Open5GS. It features a blue circular icon with a white person silhouette at the top. Below the icon, there is a 'Username' label and a text input field containing the text 'admin'. Underneath that is a 'Password' label and a text input field filled with four black dots. At the bottom of the form is a green rectangular button with the word 'Login' in white text.

← → ↻

🔒 http://localhost:9999

☰ Open5GS

+ 👤 Subscriber

📄 Profile

🔑 Account

🔍

999700000000001

999700000000001

✎ 🗑 ✕

Subscriber Configuration

🛡 46585CE8B199B49FAA5F0A2EE238A6BC
--K
E8ED289DEBA952E4283B54E88E6183CA
--OPc
8000--AMF

🛡 OPERATOR_DETERMINED_BARRING
(1)--Subscriber Status (TS 29.272
7.3.29)
--Operator Determined Barring (TS
29.272 7.3.30)

📶 1 Gbps--DL
1 Gbps--UL

SST:1 (Default S-NSSAI)

DNN/APN	Type	LBO	5QI/QCI	ARP	Capability	Vulnerability	MBR DL/UL	GBR DL/UL
internet	IPv4v6	Not allowed	9	8	Disabled	Disabled	1 Gbps / 1 Gbps	

```

test> use open5gs
switched to db open5gs
open5gs> db.subscribers.find().pretty()
[
  {
    _id: ObjectId('6a29f769214618aacf9df8a3'),
    imsi: '999700000000001',
    msisdn: [],
    imeisv: [],
    mme_host: [],
    mme_realm: [],
    purge_flag: [],
    security: {
      k: '465B5CE8B199B49FAA5F0A2EE238A6BC',
      op: null,
      opc: 'E8ED289DEBA952E4283B54E88E6183CA',
      amf: '8000'
    },
    ambr: { downlink: { value: 1, unit: 3 }, uplink: { value: 1, unit: 3 } },
    slice: [
      {
        sst: 1,
        default_indicator: true,
        session: [
          {
            name: 'internet',
            type: 3,
            qos: {
              index: 9,
              arp: {
                priority_level: 8,
                pre_emption_capability: 1,
                pre_emption_vulnerability: 1
              }
            },
            ambr: { downlink: { value: 1, unit: 3 }, uplink: { value: 1, unit: 3 } }
          }
        ]
      }
    ]
  }
]
open5gs>

```

Partie 3 — Configuration de la machine Open5GS

UPF

1. Objectif de la manipulation

Configurer la machine virtuelle chargée d'héberger l'**UPF** ou *User Plane Function*, du cœur de réseau 5G.

Contrairement à la machine Open5GS CP, cette machine ne gère ni l'authentification ni l'enregistrement du UE. Elle appartient au **plan utilisateur** et assure le transport réel des données.

L'UPF doit permettre :

- de recevoir les instructions du SMF avec le protocole PFCP ;
- de créer les règles nécessaires au traitement des paquets du UE ;
- de recevoir le trafic GTP-U envoyé par le gNB ;
- de transmettre les paquets vers l'interface réseau représentant le réseau de données.

Les deux communications principales sont :

Interface 5G	Communication	Protocole
N4	SMF vers UPF	PFCP sur UDP 8805
N3	gNB vers UPF	GTP-U sur UDP 2152

Dans cette manipulation, la machine UPF possède l'adresse :

192.168.200.129/24

L'UPF utilisera également l'interface virtuelle **ogstun**, configurée avec l'adresse :

10.45.0.1/16

2. Vérification de la configuration réseau de la machine

Afficher le nom de la machine et ses interfaces réseau :

```
Shell
hostname
```

```
Shell
ip -br address
```

Vérifier que l'interface utilisée pour les communications 5G possède l'adresse suivante :

192.168.200.129/24

Dans cette architecture, cette adresse est configurée sur l'interface ens37.

Tester ensuite la communication avec les deux autres machines :

Shell

```
ping -c 2 192.168.200.128
```

Shell

```
ping -c 2 192.168.200.130
```

3. Installer les dépendances nécessaires à Open5GS

Installer les outils de compilation et les bibliothèques requises :

Shell

```
sudo apt update
```

Shell

```
sudo apt install -y python3-pip python3-setuptools python3-wheel \
    ninja-build build-essential flex bison git cmake libsctp-dev \
    libgnutls28-dev libgcrypt-dev libssl-dev libidn11-dev \
    libmongoc-dev \
    libbson-dev libyaml-dev libnghttp2-dev libmicrohttpd-dev \
    libcurl4-gnutls-dev libtins-dev libtallic-dev meson
```

4. Compiler et installer Open5GS

Se placer dans le dossier contenant les sources d'Open5GS :

Shell

```
cd ~/open5gs-2.7.7
```

Créer le dossier de compilation avec Meson :

Shell

```
meson setup build --prefix="$(pwd)/install"
```

Compiler et installer Open5GS :

Shell

```
ninja -C build
```

Shell

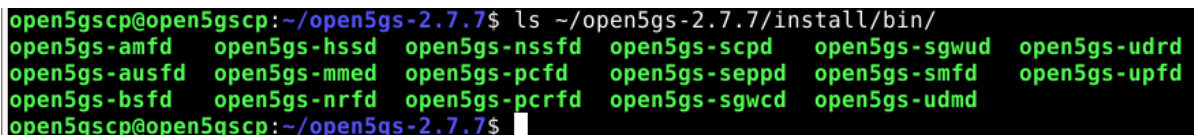
```
ninja -C build install
```

Vérifier que les exécutables ont bien été générés :

Shell

```
ls -lh ~/open5gs-2.7.7/install/bin/open5gs-*
```

Les exécutables utilisés pour le plan de contrôle doivent notamment être présents :



```
open5gscp@open5gscp:~/open5gs-2.7.7$ ls ~/open5gs-2.7.7/install/bin/  
open5gs-amfd  open5gs-hssd  open5gs-nssfd  open5gs-scpd  open5gs-sgwud  open5gs-udrd  
open5gs-ausfd  open5gs-mmed  open5gs-pcfd  open5gs-seppd  open5gs-smfd  open5gs-upfd  
open5gs-bsfd  open5gs-nrfd  open5gs-pcrfd  open5gs-sgwcd  open5gs-udmd  
open5gscp@open5gscp:~/open5gs-2.7.7$
```

Vérifier également la présence des fichiers de configuration :

Shell

```
ls -lh ~/open5gs-2.7.7/install/etc/open5gs/*.yaml
```

Les fichiers réellement utilisés se trouvent dans :

~/open5gs-2.7.7/install/etc/open5gs/

Ne pas modifier les fichiers présents dans :

~/open5gs-2.7.7/build/configs/open5gs/

Ces derniers correspondent aux fichiers utilisés pendant la compilation et ne sont pas nécessairement ceux chargés lors du lancement des fonctions.

5. Modification des fichiers UPF

a. Configurer l'UPF

Ouvrir le fichier de configuration :

```
Shell  
sudo nano ~/open5gs-2.7.7/install/etc/open5gs/upf.yaml
```

Configurer les paramètres principaux de la manière suivante :

```
XML  
upf:  
  pfcf:  
    server:  
      - address: 192.168.200.129  
  gtpu:  
    server:  
      - address: 192.168.200.129  
  session:  
    - subnet: 10.45.0.0/16  
      gateway: 10.45.0.1  
      dnn: internet
```

PFCP

```

GNU nano 8.4                                     upf.yaml *
logger:
  file:
    path: /home/open5gs/open5gs-2.7.7/install/var/log/open5gs/upf.log
# level: info # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024 # The number of UE can be increased depending on memory size.
# peer: 64

upf:
  pfcf:
    server:
      - address: 192.168.200.129
    client:
      smf: # UPF PFCF Client try to associate SMF PFCF Server
      - address: 127.0.0.4
  gtpu:
    server:
      - address: 192.168.200.129
  session:
    - subnet: 10.45.0.0/16
      gateway: 10.45.0.1
    - subnet: 2001:db8:cafe::/48
      gateway: 2001:db8:cafe::1
  metrics:
    server:
      - address: 127.0.0.7
        port: 9090

#####
# PFCF Server
#####
# o Override PFCF address to be advertised to SMF in PFCF association
# pfcf:
#   server:
#     - dev: eth0
#       advertise: open5gs-upf.svc.local
#
#####
# GTP-U Server
#####

```

b. Explication de la partie PFCF

```
pfcf:
  server:
    - address: 192.168.200.129
```

Cette adresse permet à l'UPF de recevoir les instructions du SMF.

PFCF utilise :

UDP 8805

Le SMF envoie notamment des messages permettant de :

- créer une session ;
- modifier une session ;
- supprimer une session ;
- vérifier que l'UPF reste joignable avec des Heartbeats.

Le SMF doit être configuré pour contacter :

192.168.200.129:8805

c. Explication de la partie GTP-U

gtpu:

server:

- address: 192.168.200.129

Cette adresse permet à l'UPF de recevoir le trafic utilisateur provenant du gNB.

GTP-U utilise :

UDP 2152

Les paquets IP du UE sont encapsulés dans un tunnel entre :

gNB : 192.168.200.130

UPF : 192.168.200.129

d. Explication du réseau des UE

session:

- subnet: 10.45.0.0/16

gateway: 10.45.0.1

Ce réseau est utilisé pour attribuer des adresses IP aux UE.

Dans cette architecture :

UPF / ogstun : 10.45.0.1

UE simulé : 10.45.0.3

Le réseau et la passerelle doivent correspondre aux valeurs définies dans smf.yaml.

6. Créer l'interface virtuelle ogstun

L'UPF utilise une interface TUN appelée ogstun.

Cette interface représente la sortie du réseau mobile vers le réseau de données.

Vérifier tout d'abord la présence du module TUN :

Shell

```
ls -l /dev/net/tun
```

Charger le module si nécessaire :

```
Shell  
sudo modprobe tun
```

Créer ensuite l'interface ogstun :

```
Shell  
sudo ip tuntap add name ogstun mode tun
```

Si l'interface existe déjà, la commande peut renvoyer une erreur indiquant que le fichier existe. Dans ce cas, ne pas la recréer.

Configurer son adresse :

```
Shell  
sudo ip addr replace 10.45.0.1/16 dev ogstun
```

Activer l'interface :

```
Shell  
sudo ip link set ogstun up
```

Vérifier le résultat :

```
Shell  
ip -br address show ogstun
```

Le résultat attendu est :

ogstun UNKNOWN 10.45.0.1/16

L'état UNKNOWN est normal pour une interface virtuelle de type TUN. Il ne signifie pas que l'interface est désactivée.

Vérifier également la route créée :

```
Shell  
ip route | grep 10.45
```

7. Configurer le transfert IPv4

Activer le transfert IPv4 :

```
Shell
sudo sysctl -w net.ipv4.ip_forward=1
```

Rendre la modification persistante :

```
Shell
echo "net.ipv4.ip_forward=1" | sudo tee
/etc/sysctl.d/99-open5gs-upf.conf
```

Appliquer la configuration :

```
Shell
sudo sysctl --system
```

Vérifier :

```
Shell
sysctl net.ipv4.ip_forward
```

Le résultat attendu est :

net.ipv4.ip_forward = 1

Le transfert IPv4 permet à l'UPF d'acheminer les paquets entre le réseau des UE et un autre réseau.

Partie 4 — Mise en service et validation d'une session utilisateur 5G

1. Objectif de la manipulation

Mettre en service les trois machines précédemment configurées afin de vérifier le fonctionnement complet de l'architecture 5G.

Cette manipulation devra permettre de valider successivement :

- le démarrage des fonctions du plan de contrôle Open5GS ;
- l'association PFCP entre le SMF et l'UPF ;
- la connexion du gNB à l'AMF ;
- l'authentification et l'enregistrement du UE ;
- la création d'une session PDU ;
- l'attribution d'une adresse IP au UE ;
- le transport des données entre le UE et l'UPF.

Afin de faciliter l'observation des messages, ouvrir plusieurs terminaux et conserver chaque processus au premier plan.

L'ordre de démarrage devra être respecté :

1. MongoDB
2. Fonctions du plan de contrôle Open5GS
3. UPF
4. gNB
5. UE

2. Démarrer MongoDB

Sur la machine **Open5GS CP**, vérifier que MongoDB est actif :

```
Shell
sudo systemctl start mongod
```

```
Shell
systemctl is-active mongod
```

Le résultat attendu est :

active

MongoDB doit être disponible avant les fonctions d'authentification, car il contient le profil de l'abonné.

3. Démarrer les fonctions du plan de contrôle

Sur la machine **Open5GS CP**, ouvrir un terminal différent pour chaque fonction.

Terminal CP 1 — NRF

```
Shell
cd /home/open5gscp/open5gs-2.7.7
```

```
Shell
./install/bin/open5gs-nrfd -c ./install/etc/open5gs/nrf.yaml
```

Le NRF doit être démarré en premier, car les autres fonctions s'y enregistrent.

Terminal CP 2 — UDR

```
Shell
cd /home/open5gscp/open5gs-2.7.7
```

```
Shell
./install/bin/open5gs-udrd -c ./install/etc/open5gs/udr.yaml
```

L'UDR permet l'accès aux informations stockées dans MongoDB.

Terminal CP 3 — UDM

Shell

```
cd /home/open5gscp/open5gs-2.7.7
```

Shell

```
./install/bin/open5gs-udmd -c ./install/etc/open5gs/udm.yaml
```

L'UDM gère les données d'identité et d'abonnement.

Terminal CP 4 — AUSF

Shell

```
cd /home/open5gscp/open5gs-2.7.7
```

Shell

```
./install/bin/open5gs-ausfd -c ./install/etc/open5gs/ausf.yaml
```

L'AUSF participe à l'authentification du UE.

Terminal CP 5 — PCF

Shell

```
cd /home/open5gscp/open5gs-2.7.7
```

Shell

```
./install/bin/open5gs-pcfd -c ./install/etc/open5gs/pcf.yaml
```

Le PCF fournit les politiques appliquées aux sessions.

Terminal CP 6 — NSSF

Shell

```
cd /home/open5gscp/open5gs-2.7.7
```

Shell

```
./install/bin/open5gs-nssfd -c ./install/etc/open5gs/nssf.yaml
```


Le NSSF intervient dans la sélection du slice SST 1.

Terminal CP 7 — SMF

Shell

```
cd /home/open5gscp/open5gs-2.7.7
```

Shell

```
./install/bin/open5gs-smfd -c ./install/etc/open5gs/smf.yaml
```

Le SMF gère les sessions PDU et programme l'UPF avec PFCP.

Terminal CP 8 — AMF

Shell

```
cd /home/open5gscp/open5gs-2.7.7
```

Shell

```
./install/bin/open5gs-amfd -c ./install/etc/open5gs/amf.yaml
```

L'AMF reçoit la connexion du gNB et gère l'enregistrement du UE.

```
open5gscp@open5gscp:~$ systemctl is-active mongod
active
open5gscp@open5gscp:~$ pgrep -af open5gs-
2414 /home/open5gscp/open5gs-2.7.7/install/bin/open5gs-nrfd -c /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/nrf.yaml
2435 /home/open5gscp/open5gs-2.7.7/install/bin/open5gs-scpd -c /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/scp.yaml
2456 /home/open5gscp/open5gs-2.7.7/install/bin/open5gs-ausfd -c /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/ausf.yaml
2457 /home/open5gscp/open5gs-2.7.7/install/bin/open5gs-udmd -c /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/udm.yaml
2458 /home/open5gscp/open5gs-2.7.7/install/bin/open5gs-udrd -c /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/udr.yaml
2459 /home/open5gscp/open5gs-2.7.7/install/bin/open5gs-pcfd -c /home/open5gscp/open5gs-2.7.7/install/etc/open5gs/pcf.yaml
3511 sudo ./install/bin/open5gs-nssf -c ./install/etc/open5gs/nssf.yaml
3513 sudo ./install/bin/open5gs-nssf -c ./install/etc/open5gs/nssf.yaml
3514 ./install/bin/open5gs-nssf -c ./install/etc/open5gs/nssf.yaml
3527 sudo ./install/bin/open5gs-smfd -c ./install/etc/open5gs/smf.yaml
3529 sudo ./install/bin/open5gs-smfd -c ./install/etc/open5gs/smf.yaml
3530 ./install/bin/open5gs-smfd -c ./install/etc/open5gs/smf.yaml
5161 tail -n 0 -F /home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/amf.log /home/open5gscp/open5gs-2.7.7/install/var/log/
open5gs-2.7.7/install/var/log/open5gs/udm.log /home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/udr.log
5333 sudo ./install/bin/open5gs-amfd -c ./install/etc/open5gs/amf.yaml
5337 sudo ./install/bin/open5gs-amfd -c ./install/etc/open5gs/amf.yaml
5338 ./install/bin/open5gs-amfd -c ./install/etc/open5gs/amf.yaml
open5gscp@open5gscp:~$ sudo ss -ltnA sctp | grep 38412
[sudo] Mot de passe de open5gscp :
LISTEN 0 5 192.168.200.128:38412 0.0.0.0:* users:(("open5gs-amfd",pid=5338,fd=8))
open5gscp@open5gscp:~$ sudo ss -ltnp | grep 8805
UNCONN 0 0 192.168.200.128:8805 0.0.0.0:* users:(("open5gs-smfd",pid=3530,fd=19))
```

4. Préparer et démarrer l'UPF

Sur la machine **Open5GS UPF**, vérifier l'interface ogstun :

```
Shell
sudo /usr/local/sbin/configure-ogstun.sh
```

Exécuter :

```
Shell
sudo modprobe tun
```

```
Shell
sudo ip tuntap add name ogstun mode tun 2>/dev/null || true
```

```
Shell
sudo ip addr replace 10.45.0.1/16 dev ogstun
```

```
Shell
sudo ip link set ogstun up
```

Vérifier :

```
Shell
ip -br address show ogstun
```

```
Shell
ip route | grep 10.45
```

Le résultat attendu est :

```
ogstun UNKNOWN 10.45.0.1/16
10.45.0.0/16 dev ogstun
```

Ouvrir ensuite un terminal dédié à l'UPF :

```
Shell
cd /home/open5gs/open5gs-2.7.7
```

Shell

```
sudo ./install/bin/open5gs-upfd -c ./install/etc/open5gs/upf.yaml
```

Dans un second terminal, vérifier les ports :

Shell

```
pgrep -a -x open5gs-upfd
```

Shell

```
sudo ss -ltnp | grep -E ':8805|:2152'
```

Les résultats attendus sont :

192.168.200.129:8805

192.168.200.129:2152

Le port 8805 correspond à PFCP.

Le port 2152 correspond à GTP-U.

```
open5gs@open5gs:~$ pgrep -a open5gs-upfd
3064 ./install/bin/open5gs-upfd -c ./install/etc/open5gs/upf.yaml
open5gs@open5gs:~$ sudo ss -ltnp | grep 8805
[sudo] Mot de passe de open5gs :
UNCONN 0      0      192.168.200.129:8805      0.0.0.0:*      users: (("open5gs-upfd",pid=3064,fd=8))
open5gs@open5gs:~$ sudo ss -ltnp | grep 2152
UNCONN 0      0      192.168.200.129:2152      0.0.0.0:*      users: (("open5gs-upfd",pid=3064,fd=9))
open5gs@open5gs:~$ ip -br address show ogstun
ogstun      UNKNOWN      10.45.0.1/16 fe80::91f4:885a:621:5ca3/64
open5gs@open5gs:~$ ip route | grep 10.45
10.45.0.0/16 dev ogstun proto kernel scope link src 10.45.0.1
```

5. Vérifier l'association PFCP entre le SMF et l'UPF

Après le démarrage du SMF et de l'UPF, observer le trafic PFCP :

Shell

```
sudo tcpdump -ni ens37 udp port 8805
```

Des échanges doivent apparaître entre :

192.168.200.128:8805

et

192.168.200.129:8805

À ce stade, les messages observés sont principalement des Heartbeats PFCP.

Ils permettent au SMF et à l'UPF de vérifier que leur pair est toujours disponible.

```
ueransim@ueransim:~$ ip -br address show uesimtun0
uesimtun0          UNKNOWN          10.45.0.8/24 fe80::7cc8:da52:bf2b:8e41/64
ueransim@ueransim:~$ ping -I uesimtun0 -c 4 10.45.0.1
PING 10.45.0.1 (10.45.0.1) from 10.45.0.8 uesimtun0: 56(84) bytes of data.
64 bytes from 10.45.0.1: icmp_seq=1 ttl=64 time=6.02 ms
64 bytes from 10.45.0.1: icmp_seq=2 ttl=64 time=4.25 ms
64 bytes from 10.45.0.1: icmp_seq=3 ttl=64 time=4.03 ms
64 bytes from 10.45.0.1: icmp_seq=4 ttl=64 time=3.35 ms

--- 10.45.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 3.346/4.410/6.024/0.988 ms
```

```
open5gs@open5gs:~$ sudo tcpdump -ni ens37 udp port 8805
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens37, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:51:42.263423 IP 192.168.200.129.8805 > 192.168.200.128.8805: UDP, length 16
15:51:42.264194 IP 192.168.200.128.8805 > 192.168.200.129.8805: UDP, length 16
15:51:42.264415 IP 192.168.200.128.8805 > 192.168.200.129.8805: UDP, length 16
15:51:42.264515 IP 192.168.200.129.8805 > 192.168.200.128.8805: UDP, length 16
^C
4 packets captured
4 packets received by filter
0 packets dropped by kernel
```

6. Démarrer le gNB

Sur la machine **UERANSIM**, ouvrir un premier terminal :

```
Shell
cd /opt/UERANSIM
```

```
Shell
sudo ./build/nr-gnb -c config/open5gs-gnb.yaml
```

Attendre les messages suivants :

SCTP connection established

NG Setup procedure is successful

```
ueransim@ueransim:/opt/UERANSIM$ sudo ./build/nr-gnb -c config/open5gs-gnb.yaml
[sudo] Mot de passe de ueransim :
UERANSIM v3.2.8
[2026-06-16 19:37:36.533] [sctp] [info] Trying to establish SCTP connection... (192.168.200.128:38412)
[2026-06-16 19:37:36.543] [sctp] [info] SCTP connection established (192.168.200.128:38412)
[2026-06-16 19:37:36.544] [sctp] [debug] SCTP association setup ascId[4]
[2026-06-16 19:37:36.544] [ngap] [debug] Sending NG Setup Request
[2026-06-16 19:37:36.545] [ngap] [debug] NG Setup Response received
[2026-06-16 19:37:36.545] [ngap] [info] NG Setup procedure is successful
[2026-06-16 19:38:04.618] [rrc] [debug] UE[1] new signal detected
[2026-06-16 19:38:07.119] [rrc] [info] RRC Setup for UE[1]
[2026-06-16 19:38:07.123] [ngap] [debug] Initial NAS message received from UE[1]
[2026-06-16 19:38:07.219] [ngap] [debug] UE Context Release Command received
[2026-06-16 19:38:07.219] [rrc] [info] Releasing RRC connection for UE[1]
[2026-06-16 19:38:17.635] [rrc] [info] RRC Setup for UE[1]
```

Le premier message confirme la connexion SCTP avec l'AMF.

Le second indique que l'AMF a accepté le gNB.

7. Démarrer le UE

Conserver le terminal du gNB ouvert.

Dans un deuxième terminal sur UERANSIM, lancer le UE :

```
Shell
cd /opt/UERANSIM
```

```
Shell
sudo ./build/nr-ue -c config/open5gs-ue.yaml
```

Observer les différentes étapes :

```
^Cueransim@ueransim: /opt/UERANSIM$ sudo ./build/nr-ue -c config/open5gs-ue.yaml
UERANSIM v3.2.8
[2026-06-16 19:46:28.226] [nas] [info] UE switches to state [MM-DEREGISTERED/PLMN-SEARCH]
[2026-06-16 19:46:28.227] [rrc] [debug] New signal detected for cell[1], total [1] cells in coverage
[2026-06-16 19:46:28.228] [nas] [info] Selected plmn[999/70]
[2026-06-16 19:46:28.228] [rrc] [info] Selected cell plmn[999/70] tac[1] category[SUITABLE]
[2026-06-16 19:46:28.228] [nas] [info] UE switches to state [MM-DEREGISTERED/PS]
[2026-06-16 19:46:28.228] [nas] [info] UE switches to state [MM-DEREGISTERED/NORMAL-SERVICE]
[2026-06-16 19:46:28.228] [nas] [debug] Initial registration required due to [MM-DEREG-NORMAL-SERVICE]
[2026-06-16 19:46:28.234] [nas] [debug] UAC access attempt is allowed for identity[0], category[M0_sig]
[2026-06-16 19:46:28.234] [nas] [debug] Sending Initial Registration
[2026-06-16 19:46:28.234] [rrc] [debug] Sending RRC Setup Request
[2026-06-16 19:46:28.234] [nas] [info] UE switches to state [MM-REGISTER-INITIATED]
[2026-06-16 19:46:28.235] [rrc] [info] RRC connection established
[2026-06-16 19:46:28.235] [rrc] [info] UE switches to state [RRC-CONNECTED]
[2026-06-16 19:46:28.235] [nas] [info] UE switches to state [CM-CONNECTED]
[2026-06-16 19:46:28.257] [nas] [debug] Authentication Request received
[2026-06-16 19:46:28.257] [nas] [debug] Received SQN [000000000101]
[2026-06-16 19:46:28.257] [nas] [debug] SQN-MS [000000000000]
[2026-06-16 19:46:28.265] [nas] [debug] Security Mode Command received
[2026-06-16 19:46:28.266] [nas] [debug] Selected integrity[2] ciphering[0]
[2026-06-16 19:46:28.277] [nas] [debug] Registration accept received
[2026-06-16 19:46:28.277] [nas] [info] UE switches to state [MM-REGISTERED/NORMAL-SERVICE]
[2026-06-16 19:46:28.277] [nas] [debug] Sending Registration Complete
[2026-06-16 19:46:28.277] [nas] [info] Initial Registration is successful
[2026-06-16 19:46:28.277] [nas] [debug] Sending PDU Session Establishment Request
[2026-06-16 19:46:28.278] [nas] [debug] UAC access attempt is allowed for identity[0], category[M0_sig]
[2026-06-16 19:46:28.480] [nas] [debug] Configuration Update Command received
[2026-06-16 19:46:28.500] [nas] [debug] PDU Session Establishment Accept received
[2026-06-16 19:46:28.500] [nas] [info] PDU Session establishment is successful PSI[1]
[2026-06-16 19:46:28.529] [app] [info] Connection setup for PDU session[1] is successful, TUN interface[uesimtun0, 10.45.0.3] is up.
```

8. Enregistrement du UE

Authentication Request

Security Mode Command

Registration Accept

Le cœur vérifie l'identité du UE, active la sécurité NAS puis accepte son enregistrement.

9. Création de la session PDU

PDU Session Establishment Accept

Connection setup for PDU session is successful

Le SMF crée la session et programme l'UPF avec PFCP.

L'interface **uesimtun0** est ensuite créée avec l'adresse **10.45.0.3**

10. Confirmer l'enregistrement dans les journaux de l'AMF

Sur Open5GS CP, exécuter :

Shell

```
LOG=/home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/amf.log
```

Shell

```
grep 'Registration complete'  
/home/open5gscp/open5gs-2.7.7/install/var/log/open5gs/amf.log |  
tail
```

Les lignes importantes sont :

Registration complete

Number of AMF-Sessions is now 1

UE SUPI[imsi-99970000000001]

DNN[internet]

La ligne Registration complete confirme que l'UE a terminé son enregistrement.

La présence d'une session AMF et du DNN internet confirme qu'une demande de session PDU a été traitée.

11. Vérifier l'adresse IP et la connectivité du UE

Sur UERANSIM, vérifier l'interface créée :

Shell

```
ip -br address show uesimtun0
```

Le résultat attendu est :

uesimtun0 UNKNOWN 10.45.0.3/24

Afficher la table de routage du UE :

```
Shell
ip route show table rt_uesimtun0
```

Tester ensuite la communication avec l'UPF :

```
Shell
ping -I uesimtun0 -c 4 10.45.0.1
```

Ce test valide le chemin suivant :

UE 10.45.0.3
→ interface uesimtun0
→ gNB
→ tunnel GTP-U
→ UPF
→ interface ogstun 10.45.0.1

AMF :

```
'Registration complete|AMF-Sessions|UE SUPI|DNN[Internet]|nsmf_pdusession' \
"$LOG" | tail -n 12
06/16 19:44:17.482: [gmm] INFO: [imsi-999700000000001] Registration complete (./src/amf/gmm-sm.c:3146)
06/16 19:44:17.485: [amf] INFO: [Added] Number of AMF-Sessions is now 1 (./src/amf/context.c:2798)
06/16 19:44:17.485: [gmm] INFO: UE SUPI[imsi-999700000000001] DNN[Internet] LB0[0] S_NSSAI[SST:1 SD:0xfffff] smContextRef[NULL] smContextResourceURI[NULL] (./src/amf/gmm-handler.c:1419)
06/16 19:44:17.486: [gmm] INFO: nsmf_pdusession [1:0x559e99cb11d0:(nil)] (./src/amf/gmm-handler.c:1617)
06/16 19:46:28.258: [amf] INFO: [Removed] Number of AMF-Sessions is now 0 (./src/amf/context.c:2805)
06/16 19:46:28.486: [gmm] INFO: [imsi-999700000000001] Registration complete (./src/amf/gmm-sm.c:3146)
06/16 19:46:28.486: [amf] INFO: [Added] Number of AMF-Sessions is now 1 (./src/amf/context.c:2798)
06/16 19:46:28.486: [gmm] INFO: UE SUPI[imsi-999700000000001] DNN[Internet] LB0[0] S_NSSAI[SST:1 SD:0xfffff] smContextRef[NULL] smContextResourceURI[NULL] (./src/amf/gmm-handler.c:1419)
06/16 19:46:28.487: [gmm] INFO: nsmf_pdusession [1:0x559e99cb11d0:(nil)] (./src/amf/gmm-handler.c:1617)
```

Capture du test utilisateur :

```
ueransim@ueransim:~$ ip -br address show uesimtun0
uesimtun0          UNKNOWN          10.45.0.3/24 fe80::38e4:2a7d:f8d:15ee/64
ueransim@ueransim:~$ ip route show table rt_uesimtun0
default dev uesimtun0 scope link
ueransim@ueransim:~$ ping -I uesimtun0 -c 4 10.45.0.1
PING 10.45.0.1 (10.45.0.1) from 10.45.0.3 uesimtun0: 56(84) bytes of data.
64 bytes from 10.45.0.1: icmp_seq=1 ttl=64 time=5.15 ms
64 bytes from 10.45.0.1: icmp_seq=2 ttl=64 time=2.84 ms
^C
--- 10.45.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 2.843/3.996/5.150/1.153 ms
```

Capture PFCP :

Capture tcpdump apres lancement de l'UE


```

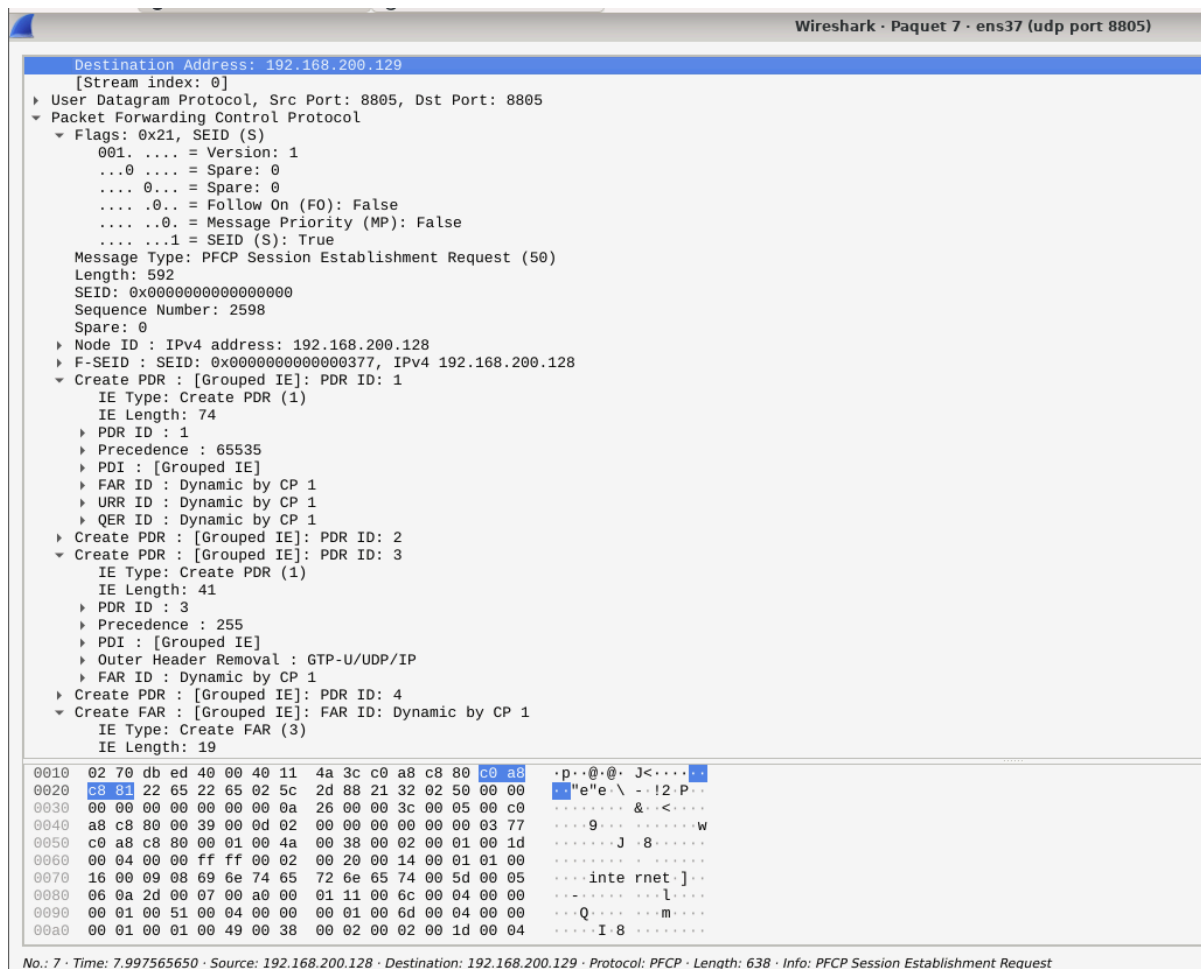
open5gscp@open5gscp:~$ sudo tcpdump -ni any -nn -vv 'udp port 8805 and host 192.168.200.129'
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
01:12:52.741411 ens37 Out IP (tos 0x0, ttl 64, id 16166, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x127d -> 0x3956!] UDP, length 16
01:12:52.742274 ens37 In IP (tos 0x0, ttl 64, id 18126, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 16
01:12:52.742275 ens37 In IP (tos 0x0, ttl 64, id 18127, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 16
01:12:52.742488 ens37 Out IP (tos 0x0, ttl 64, id 16167, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x127d -> 0xb854!] UDP, length 16
01:12:53.200291 ens37 Out IP (tos 0x0, ttl 64, id 16228, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x127d -> 0xa7e6!] UDP, length 16
01:12:53.200981 ens37 In IP (tos 0x0, ttl 64, id 18169, offset 0, flags [DF], proto UDP (17), length 161)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 133
01:12:53.450884 ens37 Out IP (tos 0x0, ttl 64, id 16245, offset 0, flags [DF], proto UDP (17), length 624)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x14c1 -> 0x7742!] UDP, length 596
01:12:53.452201 ens37 In IP (tos 0x0, ttl 64, id 18187, offset 0, flags [DF], proto UDP (17), length 144)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 116
01:12:53.459350 ens37 Out IP (tos 0x0, ttl 64, id 16247, offset 0, flags [DF], proto UDP (17), length 103)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x12b8 -> 0x3a96!] UDP, length 75
01:12:53.460452 ens37 In IP (tos 0x0, ttl 64, id 18189, offset 0, flags [DF], proto UDP (17), length 49)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 21
01:13:03.745035 ens37 In IP (tos 0x0, ttl 64, id 20387, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 16
01:13:03.745073 ens37 Out IP (tos 0x0, ttl 64, id 18358, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x127d -> 0x3556!] UDP, length 16
01:13:03.745197 ens37 Out IP (tos 0x0, ttl 64, id 18359, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.128.8805 > 192.168.200.129.8805: [bad udp cksum 0x127d -> 0xb754!] UDP, length 16
01:13:03.745725 ens37 In IP (tos 0x0, ttl 64, id 20388, offset 0, flags [DF], proto UDP (17), length 44)
    192.168.200.129.8805 > 192.168.200.128.8805: [udp sum ok] UDP, length 16

```

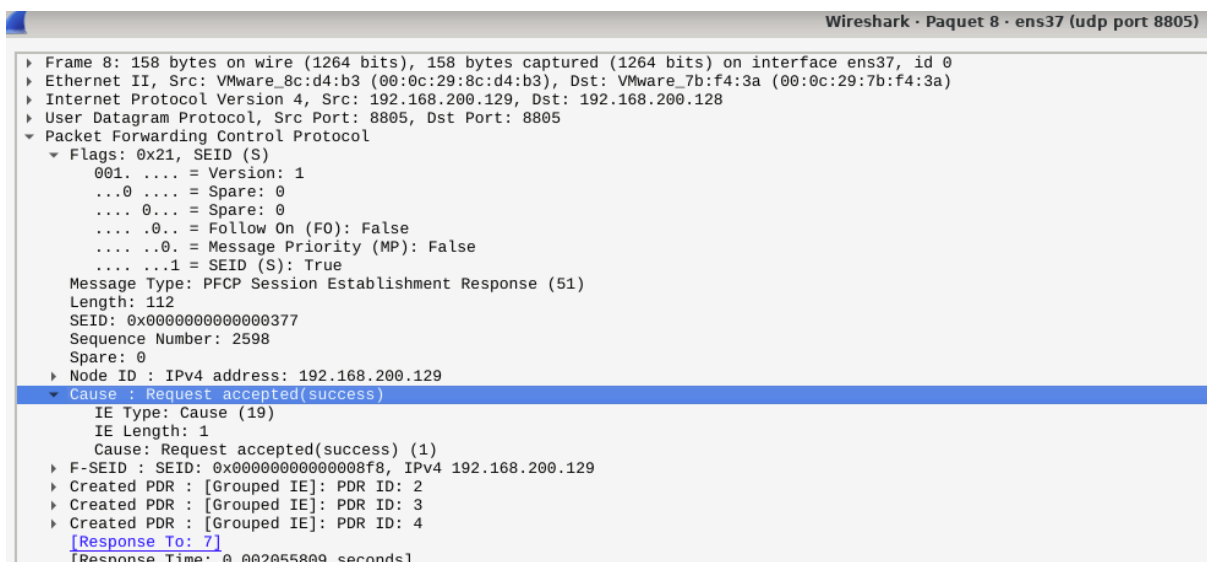
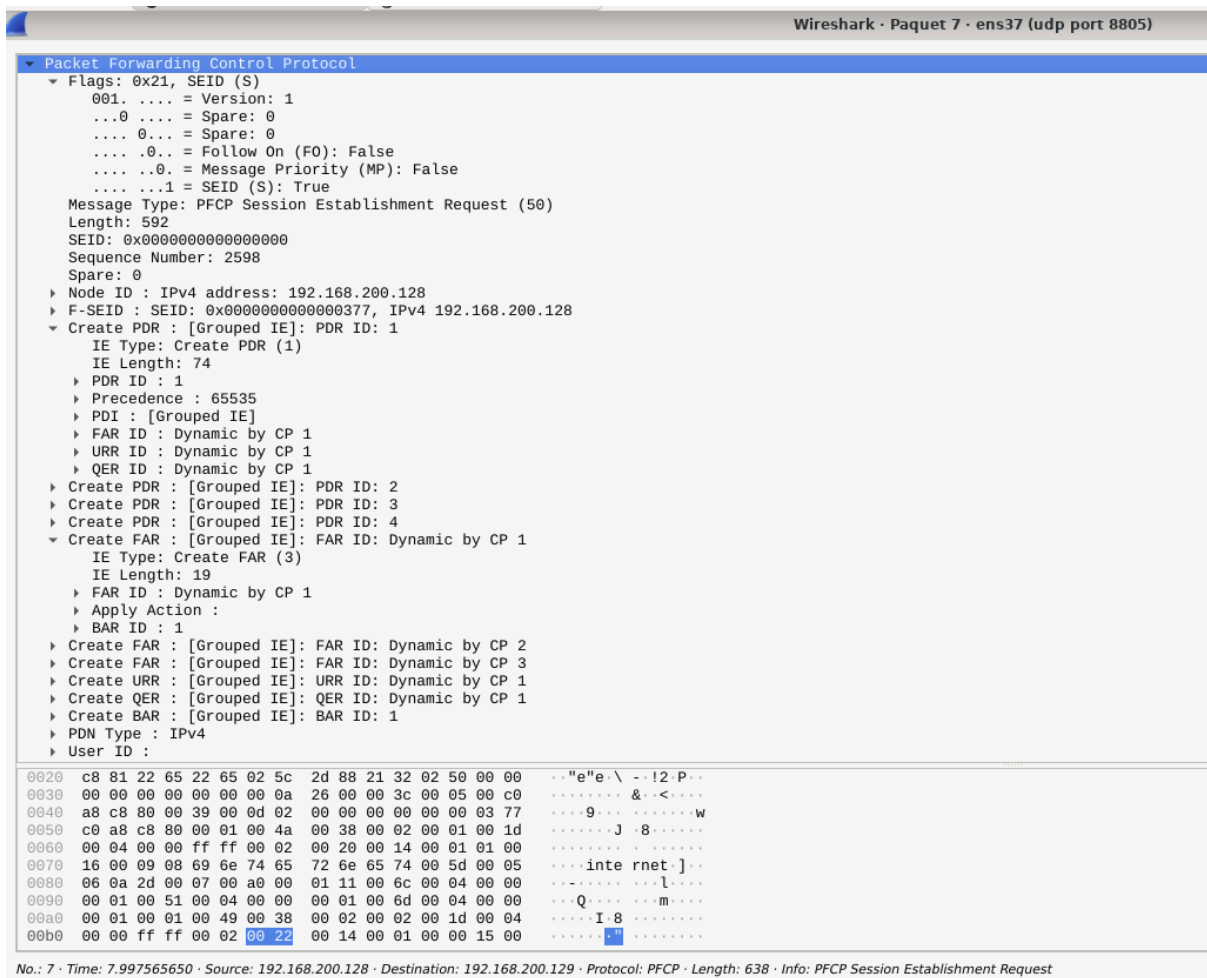
Wireshark :

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.200.128	192.168.200.129	PFCP	60	PFCP Heartbeat Request
2	0.000291367	192.168.200.129	192.168.200.128	PFCP	58	PFCP Heartbeat Response
3	0.000386836	192.168.200.129	192.168.200.128	PFCP	58	PFCP Heartbeat Request
4	0.001150739	192.168.200.128	192.168.200.129	PFCP	60	PFCP Heartbeat Response
5	7.745508230	192.168.200.128	192.168.200.129	PFCP	60	PFCP Session Deletion Request
6	7.746234805	192.168.200.129	192.168.200.128	PFCP	175	PFCP Session Deletion Response
7	7.997565650	192.168.200.128	192.168.200.129	PFCP	638	PFCP Session Establishment Request
8	7.999621459	192.168.200.129	192.168.200.128	PFCP	158	PFCP Session Establishment Response
9	8.000817288	192.168.200.128	192.168.200.129	PFCP	117	PFCP Session Modification Request
10	8.010042200	192.168.200.129	192.168.200.128	PFCP	63	PFCP Session Modification Response
11	11.004989286	192.168.200.128	192.168.200.129	PFCP	60	PFCP Heartbeat Request
12	11.005386282	192.168.200.129	192.168.200.128	PFCP	58	PFCP Heartbeat Response
13	11.005285600	192.168.200.129	192.168.200.128	PFCP	58	PFCP Heartbeat Request
14	11.005966774	192.168.200.128	192.168.200.129	PFCP	60	PFCP Heartbeat Response
15	22.010434711	192.168.200.129	192.168.200.128	PFCP	58	PFCP Heartbeat Request
16	22.010916166	192.168.200.128	192.168.200.129	PFCP	60	PFCP Heartbeat Request
17	22.011000374	192.168.200.129	192.168.200.128	PFCP	58	PFCP Heartbeat Response
18	22.011103709	192.168.200.128	192.168.200.129	PFCP	60	PFCP Heartbeat Response

Cette capture présente la chronologie des échanges PFCP entre le SMF, situé à l'adresse 192.168.200.128, et l'UPF, situé à l'adresse 192.168.200.129. Les premiers messages sont des requêtes et réponses Heartbeat, utilisées pour vérifier périodiquement que les deux fonctions restent joignables. Une ancienne session est ensuite supprimée au moyen d'un PFCP Session Deletion Request et de sa réponse. Le SMF demande alors la création d'une nouvelle session avec un PFCP Session Establishment Request, accepté par l'UPF. Enfin, une modification de session est réalisée afin de compléter les paramètres nécessaires au transfert des données. Cette séquence représente le fonctionnement normal de PFCP lors de la reconnexion d'un UE.

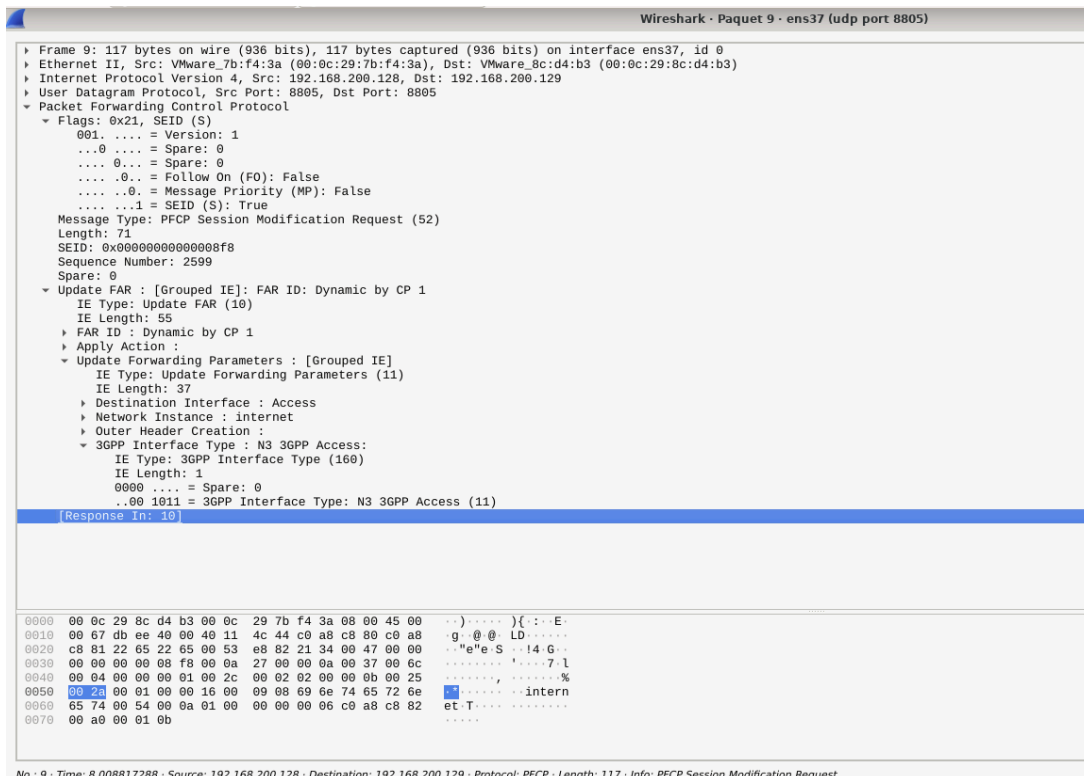


Cette capture détaille le message PFCP Session Establishment Request, de type 50, envoyé par le SMF vers l'UPF. Les deux fonctions communiquent sur le port UDP 8805, correspondant à l'interface N4. Le message possède le numéro de séquence 2598, qui permettra de l'associer à sa réponse. Le Node ID et le CP F-SEID identifient le SMF et la partie contrôle de la session. Le SEID présent dans l'en-tête vaut encore zéro, car l'UPF n'a pas encore attribué son propre identifiant à cette nouvelle session. Le SMF transmet ainsi à l'UPF toutes les informations nécessaires à sa création.



Cette capture présente la réponse envoyée par l'UPF au SMF après le traitement de la demande de création. Le message est un PFCP Session Establishment Response, de type 51. Son numéro de séquence est également 2598, ce qui permet de confirmer qu'il répond à la requête précédente. Le champ Cause: Request accepted (success) indique que l'UPF a

accepté et installé les règles demandées. L'UPF retourne également son propre F-SEID, associé à l'adresse 192.168.200.129, ainsi que plusieurs Created PDR. La session PFCP est donc désormais créée à la fois du côté du SMF et du côté de l'UPF.



Cette capture montre un PFCP Session Modification Request, de type 52, envoyé par le SMF après la création initiale de la session. Le message utilise le SEID 0x00000000000008f8, qui correspond à l'identifiant attribué précédemment par l'UPF. Le SMF demande notamment une mise à jour de la règle FAR. Le champ Destination Interface: Access indique que les paquets concernés doivent être transférés vers le réseau d'accès, c'est-à-dire vers le gNB. Le champ Network Instance: internet confirme que la session utilise le DNN internet. Enfin, la présence de Outer Header Creation et de N3 3GPP Access montre que cette modification complète les paramètres nécessaires au tunnel GTP-U établi sur l'interface N3 entre le gNB et l'UPF.

- Partie attaque protocole PFCP

No.	Time	Source	Destination	Protocol	Length	Info
147	13.685500884	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
148	13.733454800	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
149	13.773608194	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
150	13.818149372	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
151	13.862385750	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
152	13.910299340	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
153	13.961817093	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
154	14.001281195	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
155	14.057569073	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
156	14.101710122	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
157	14.154065696	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
158	14.217706453	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
159	14.257992678	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
160	14.305424993	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
161	14.345790102	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
162	14.390288433	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
163	14.442382235	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
164	14.481869034	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
165	14.525446316	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
166	14.569859417	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
167	14.614565729	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request
168	14.661675176	192.168.200.150	192.168.200.129	PFCP	60	PFCP Heartbeat Request

- Frame 209: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface ens37, id 0 - Ethernet II, Src: VMware_91:bd:0b (00:0c:29:91:bd:0b), Dst: VMware_8c:d4:b3 (00:0c:29:8c:d4:b3) - Destination: VMware_8c:d4:b3 (00:0c:29:8c:d4:b3) - Source: VMware_91:bd:0b (00:0c:29:91:bd:0b) - Type: IPv4 (0x0800) [Stream index: 1] - Padding: 0000 - Internet Protocol Version 4, Src: 192.168.200.150, Dst: 192.168.200.129 - User Datagram Protocol, Src Port: 8805, Dst Port: 8805 - Source Port: 8805 - Destination Port: 8805 - Length: 24 - Checksum: 0xa5e8 [unverified] [Checksum Status: Unverified] [Stream index: 1] [Stream Packet Number: 201] [Timestamps] - UDP payload (16 bytes) - Packet Forwarding Control Protocol - Flags: 0x20 - Message Type: PFCP Heartbeat Request (1) - Length: 12	<pre> 0000 00 0c 29 8c d4 b3 0010 00 2c 00 01 00 00 0020 c8 81 22 65 22 65 0030 c9 00 00 60 00 04 </pre>
---	--

Les attaques PFCP n'ont pas été entièrement reproduites dans notre environnement. Trois scénarios théoriques ont été étudiés : flood de messages PFCP, paquets malformés et abus de gestion de session. Leurs effets potentiels seraient une augmentation de charge, des erreurs dans les journaux ou une perturbation des sessions utilisateur.

CONCLUSION TECHNIQUE

Ce projet a permis de déployer et de valider une architecture 5G simplifiée à l'aide d'Open5GS, d'UERANSIM et de plusieurs machines virtuelles.

La machine UERANSIM a permis de simuler un équipement utilisateur, appelé UE, ainsi qu'une station de base 5G, appelée gNB. Le gNB a pu établir une connexion avec l'AMF par l'interface N2, en utilisant NGAP sur SCTP.

La machine Open5GS CP a assuré les principales fonctions du plan de contrôle. L'AMF a pris en charge l'enregistrement du UE, tandis que les fonctions AUSF, UDM et UDR ont participé à son authentification grâce aux informations enregistrées dans MongoDB. Le SMF a ensuite géré la création de la session PDU.

La machine Open5GS UPF a assuré le transport des données utilisateur. Le SMF a programmé l'UPF avec le protocole PFCP sur l'interface N4. Une fois la session créée, le trafic du UE a été transporté entre le gNB et l'UPF avec GTP-U sur l'interface N3.

Le fonctionnement de l'architecture a été confirmé par plusieurs résultats :

- la connexion du gNB à l'AMF a été établie ;
- le UE a été authentifié et enregistré ;
- une session PDU a été créée pour le DNN internet ;
- l'interface uesimtun0 a reçu l'adresse 10.45.0.3 ;
- le ping vers l'interface ogstun de l'UPF en 10.45.0.1 a fonctionné ;
- les échanges PFCP entre le SMF et l'UPF ont été observés dans Wireshark.

L'analyse des captures PFCP a permis d'identifier les messages de maintien de connexion, de création, de modification et de suppression de session. Elle a également permis de mieux comprendre la manière dont le SMF transmet à l'UPF les règles nécessaires au traitement des paquets utilisateur.

Cette infrastructure constitue donc une base fonctionnelle pour poursuivre l'étude du protocole PFCP et de la sécurité de l'interface N4.

CONCLUSION GÉNÉRALE

Ce projet nous a permis de mieux comprendre le fonctionnement général d'un réseau 5G, depuis la connexion d'un équipement utilisateur jusqu'au transport de ses données.

La mise en place de trois machines virtuelles distinctes nous a permis de séparer clairement les différents rôles de l'architecture. UERANSIM a simulé l'UE et le gNB, Open5GS CP a assuré les fonctions du plan de contrôle et l'UPF a pris en charge le plan utilisateur.

Les différentes manipulations ont permis d'observer les principales étapes d'une connexion 5G : la connexion du gNB à l'AMF, l'authentification de l'abonné, son enregistrement, la création de sa session PDU et l'attribution d'une adresse IP.

L'analyse réalisée avec Wireshark a mis en évidence le rôle central de PFCP. Ce protocole permet au SMF de contrôler l'UPF en créant, modifiant ou supprimant les règles appliquées aux sessions utilisateur. Il ne transporte pas directement les données de l'UE, mais prépare leur chemin. Les données sont ensuite transportées avec GTP-U.

Nous avons également présenté plusieurs risques théoriques liés à PFCP, comme le flood de messages, les paquets malformés ou les abus de gestion de session. Ces scénarios n'ont cependant pas été complètement reproduits ni mesurés dans notre environnement. Ils constituent donc des pistes de travail pour une future étude.

Les principales améliorations possibles seraient :

- réaliser des tests d'attaque PFCP dans un environnement isolé ;
- mesurer précisément l'impact sur le processeur, la mémoire, les journaux et la connectivité du UE ;
- comparer le trafic normal et le trafic anormal ;
- mettre en place des protections comme le filtrage du port UDP 8805 et l'isolation du réseau N4.

En conclusion, le projet a permis de mettre en place une architecture 5G fonctionnelle et d'observer concrètement le rôle des principaux composants et protocoles. Il nous a surtout permis de comprendre que PFCP est un protocole essentiel au fonctionnement du plan utilisateur et qu'il doit être protégé contre les accès et les trafics non autorisés.

Références

<https://github.com/aligungr/UERANSIM/wiki/Configuration>

<https://github.com/aligungr/UERANSIM/wiki/Installation>

<https://github.com/open5gs/open5gs/commit/318eeb49a7dcdff733dec60e02d9c60aefca2fb9#diff-22a8cfd90d6bcfe5e14441a1b6b462a326a193a285430905d2d100e22d8d0292>

open5gs : <https://github.com/open5gs/open5gs/archive/v2.7.7.tar.gz>

ueransim : <https://github.com/aligungr/UERANSIM/archive/refs/tags/v3.2.7.tar.gz>

Guide de open5gs:

https://github.com/open5gs/open5gs/blob/main/docs/_docs/guide/01-quickstart.md